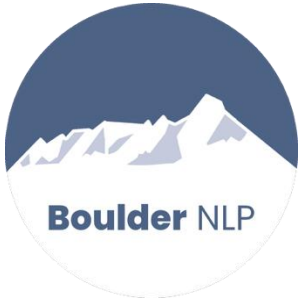
Natural Language Processing

Jim Martin -- CSCI 5832

Lecture 14



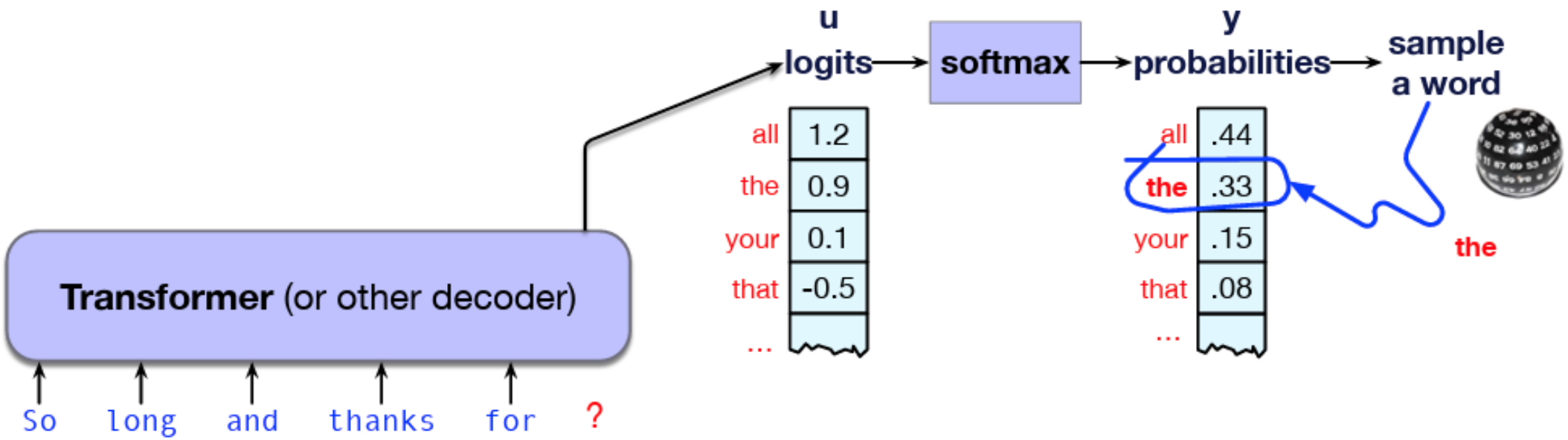
Today

- Moving Quiz 2 back a week: Tuesday 10/27
  - Chapters 5, 6, 7
    - Embeddings
    - Neural Nets
    - Transformers and Pretraining
- Sampling
- Other architectures
  - Bidirectional Transformers
  - Encoder-Decoder Transformers

Sampling

- Greedy
  - Take the top choice based on the distribution (argmax decoding).
- Random sampling from the distribution
- Temperature sampling
- Top-K sampling
- Nucleus (Top-p) sampling

Random Sampling



$$y = \text{softmax}(u)$$

Random Sampling

“One fish two fish red fish blue fish black fish blue fish”

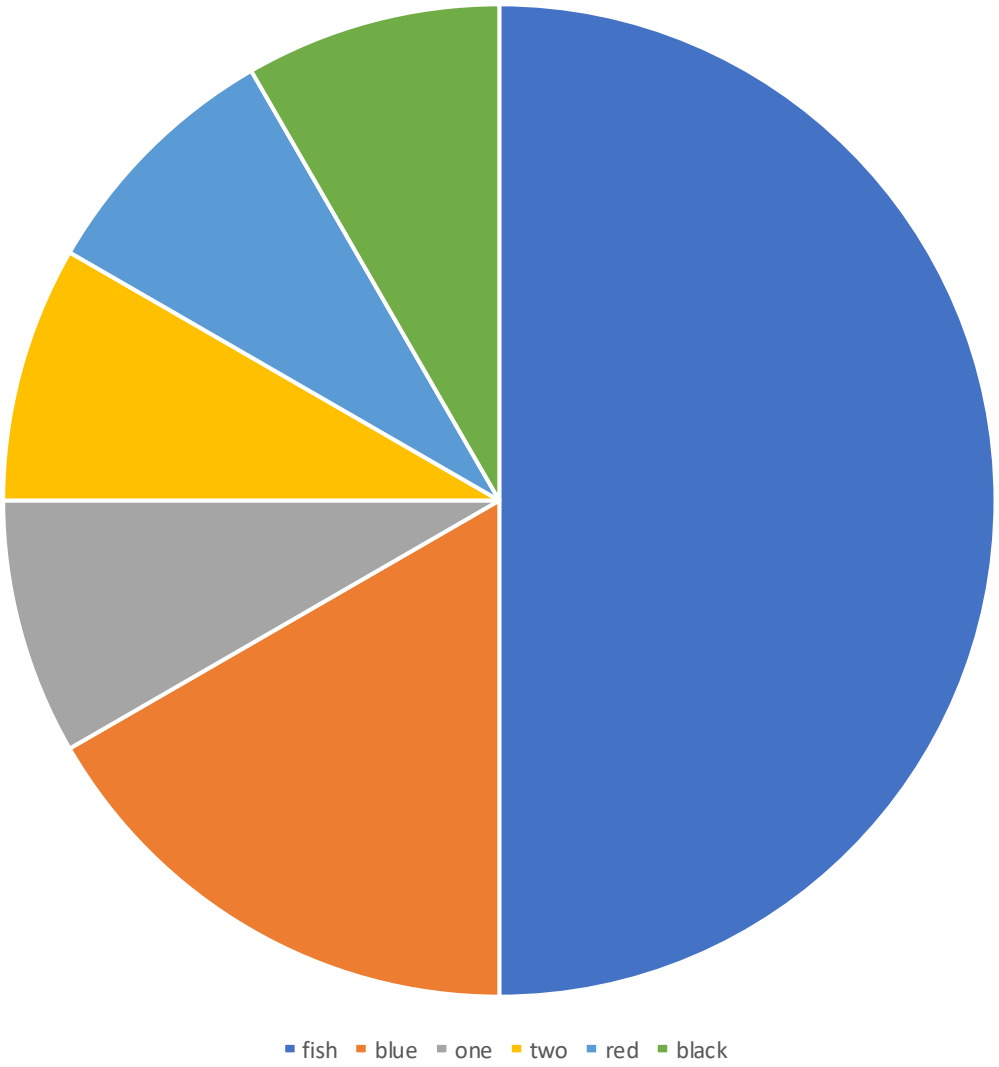
$N = 12$

$|V| = 6$

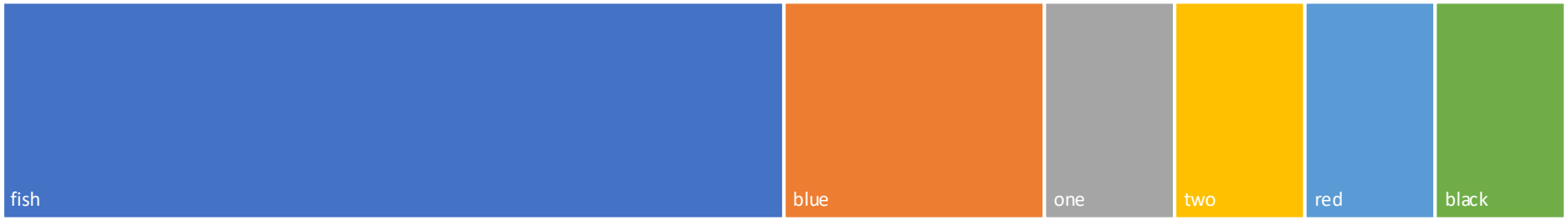
Random Sampling

- Counts
  - ”fish” = 6
  - “blue” = 2
  - ”one” = 1
  - “two” = 1
  - ”red” = 1
  - “black” = 1
- Probs
  - ”fish” = 6/12
  - “blue” = 2/12
  - ”one” = 1/12
  - “two” = 1/12
  - ”red” = 1/12
  - “black” = 1/12

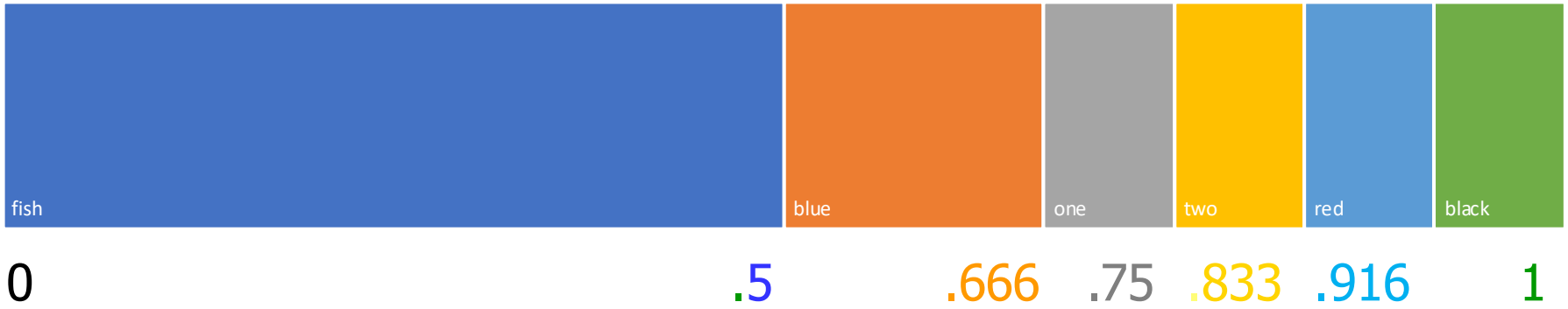
Random Sampling



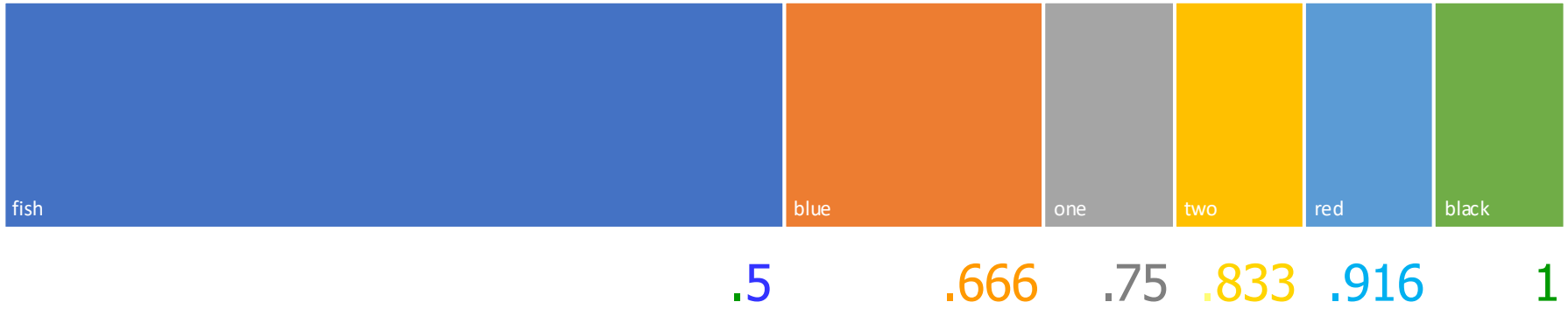
Random Sampling



Random Sampling



Random Sampling



Generate a random number between 0 and 1
Find the bin that it falls in, return the word associated with that bin
Repeat

Numpy Version

In [30]:
```
1 sample
```

Out[30]:
```
array(['fish', 'fish', 'one', 'blue', 'fish', 'fish', 'fish', 'fish',
       'one', 'one', 'blue', 'two', 'fish', 'blue', 'fish', 'fish', 'two',
       'blue', 'black', 'blue', 'fish', 'fish', 'blue', 'blue', 'fish',
       'fish', 'fish', 'fish', 'blue', 'two', 'fish', 'fish', 'one',
       'blue', 'fish', 'red', 'blue', 'fish', 'fish', 'fish', 'blue',
       'fish', 'fish', 'one', 'blue', 'red', 'red', 'blue', 'blue',
       'black', 'fish', 'fish', 'red', 'one', 'fish', 'black', 'black',
       'red', 'blue', 'fish', 'black', 'one', 'blue', 'fish', 'blue',
       'fish', 'one', 'red', 'red', 'blue', 'fish', 'blue', 'fish',
       'black', 'two', 'fish', 'fish', 'blue', 'fish', 'fish', 'black',
       'fish', 'fish', 'one', 'fish', 'fish', 'two', 'two', 'fish',
       'fish', 'fish', 'two', 'black', 'fish', 'blue', 'red', 'blue',
       'black', 'two', 'two'], dtype='<U5')
```

In [31]:
```
1 np.unique(weighted_sample, return_counts=True)
2
```

Out[31]:
```
(array(['black', 'blue', 'fish', 'one', 'red', 'two'], dtype='<U5'),
 array([ 5, 18, 53,  9,  8,  7]))
```

Problem

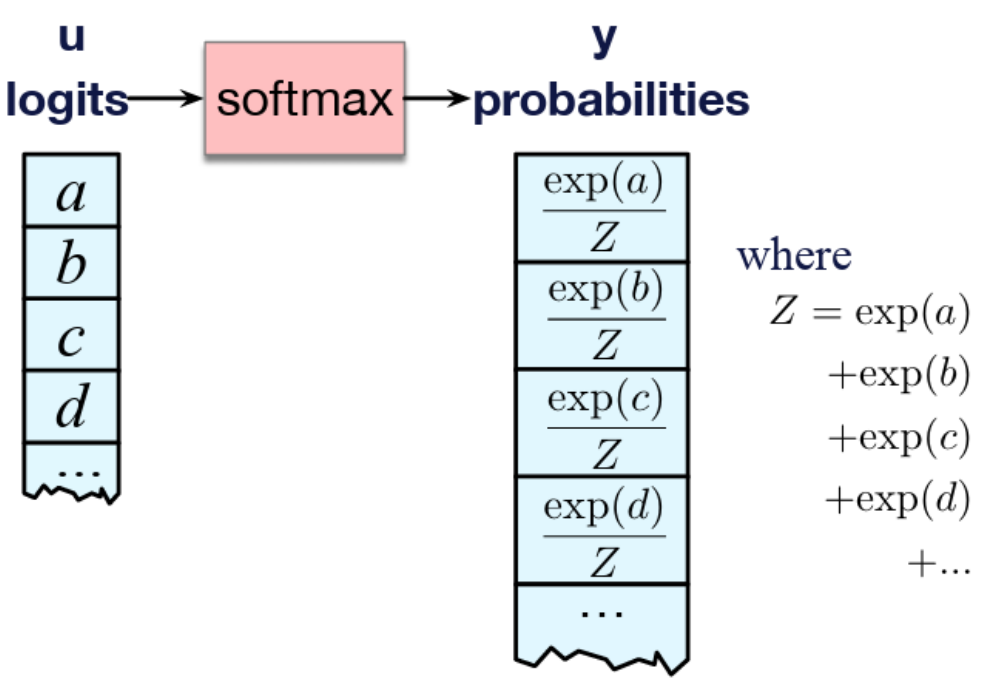
That approach produces some odd outputs since the distribution is so skewed.

Better approaches try to fix this by manipulating the distribution or restricting the sample space.

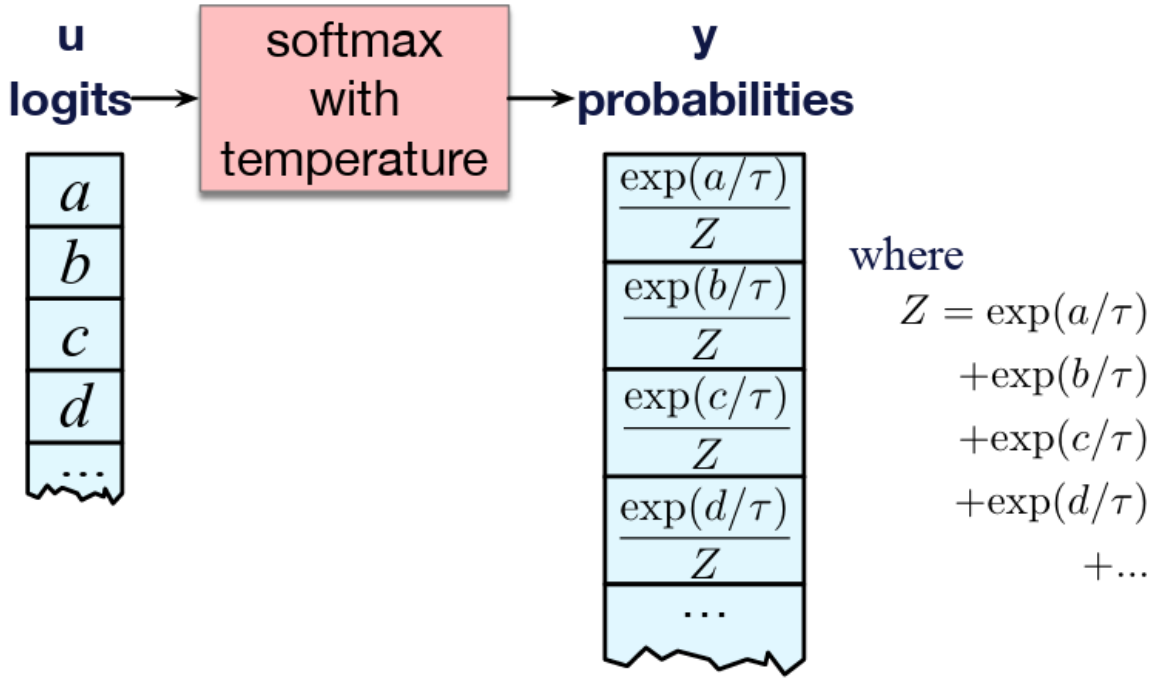
- Temperature sampling
- Top-k sampling
- Top-p sampling

Temperature

Divide the logit by a “temperature” > 0, typically between 0 and 1.

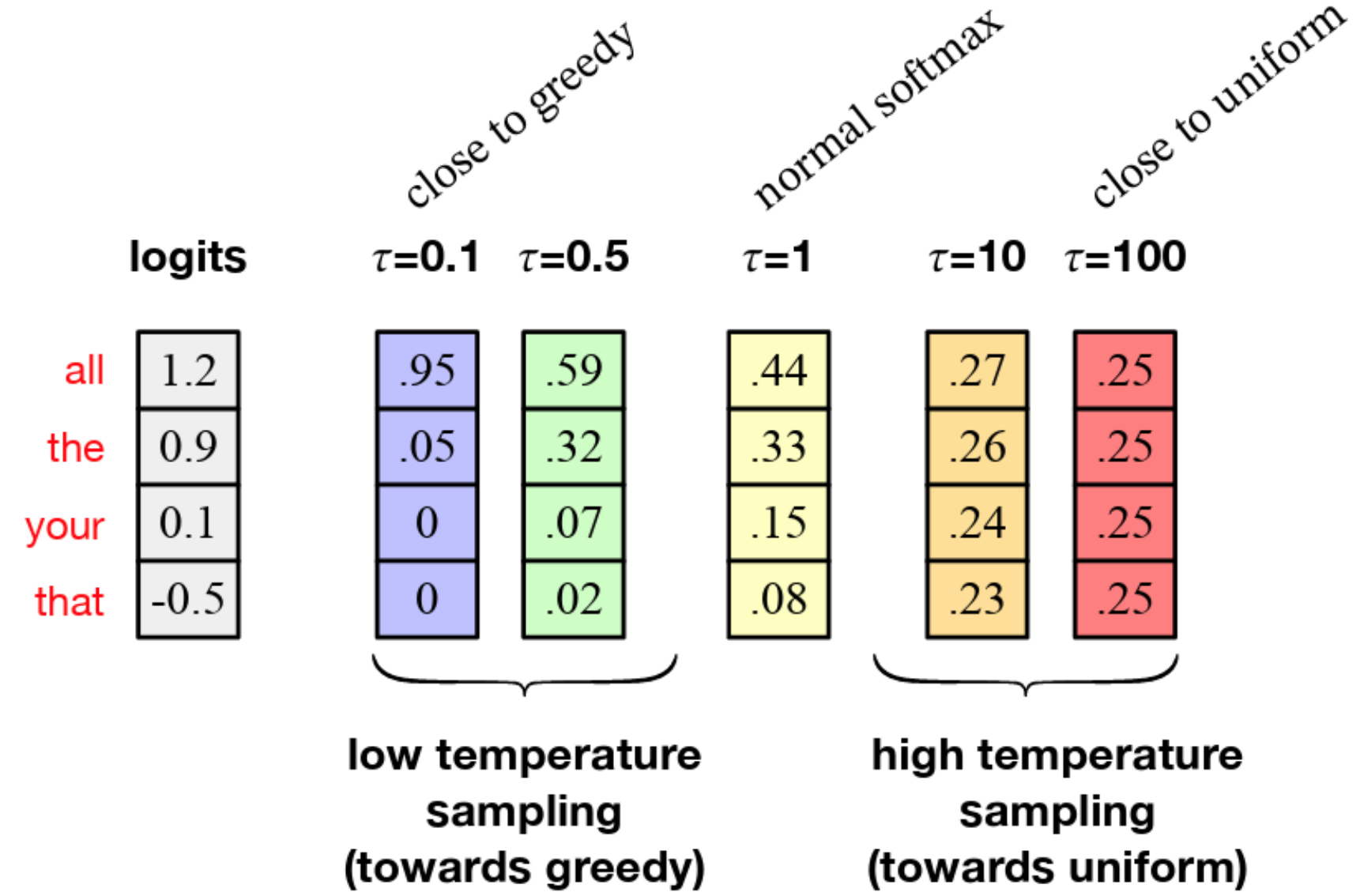


(a) $\mathbf{y} = \mathrm{softmax}(\mathbf{u})$



(b) $\mathbf{y} = \mathrm{softmax}(\mathbf{u}/\tau)$

Temperature



Top-k

1. Truncate the original V to just the top-k logits
2. Generate softmax from that
3. Sample from that distribution

Never generates a word outside the top-k

But the top-k differs with each time step

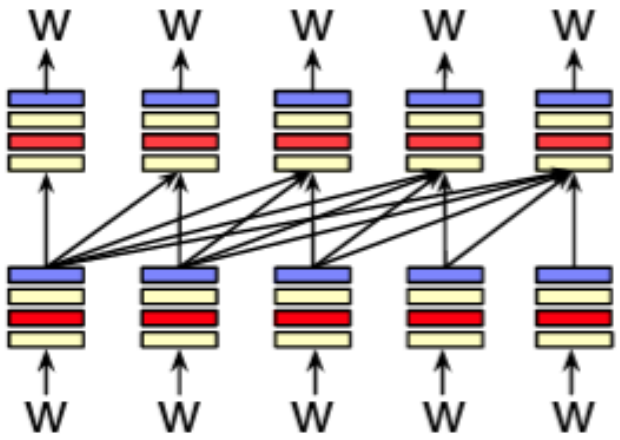
Top-p

Only keep the top p percent of the probability mass and sample from that. So instead of the sampling from the top-k terms, sample from for example, the top 50% of the probability mass.

$$\sum_{w \in V^{(p)}} P(w|\mathbf{w}_{<t}) \geq p$$

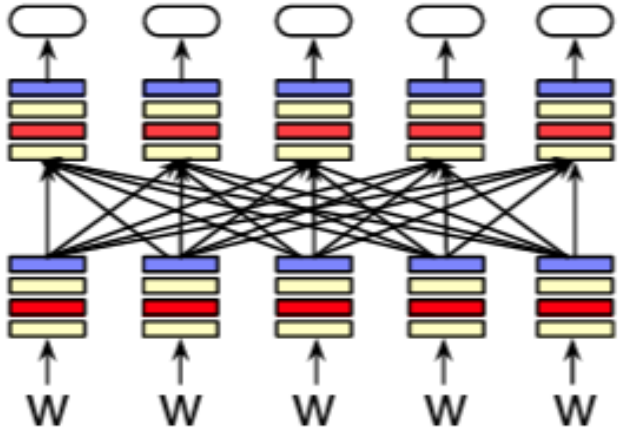
Architectures

GPT family



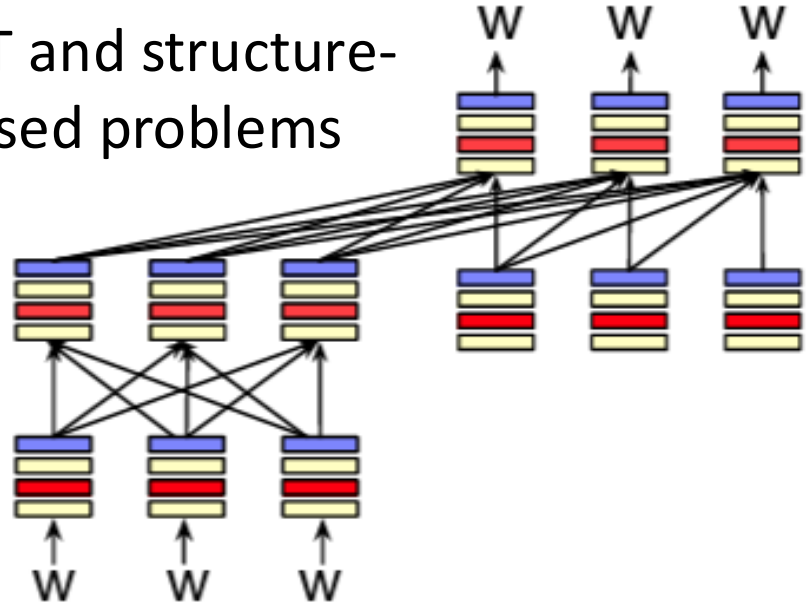
Decoder

BERT family



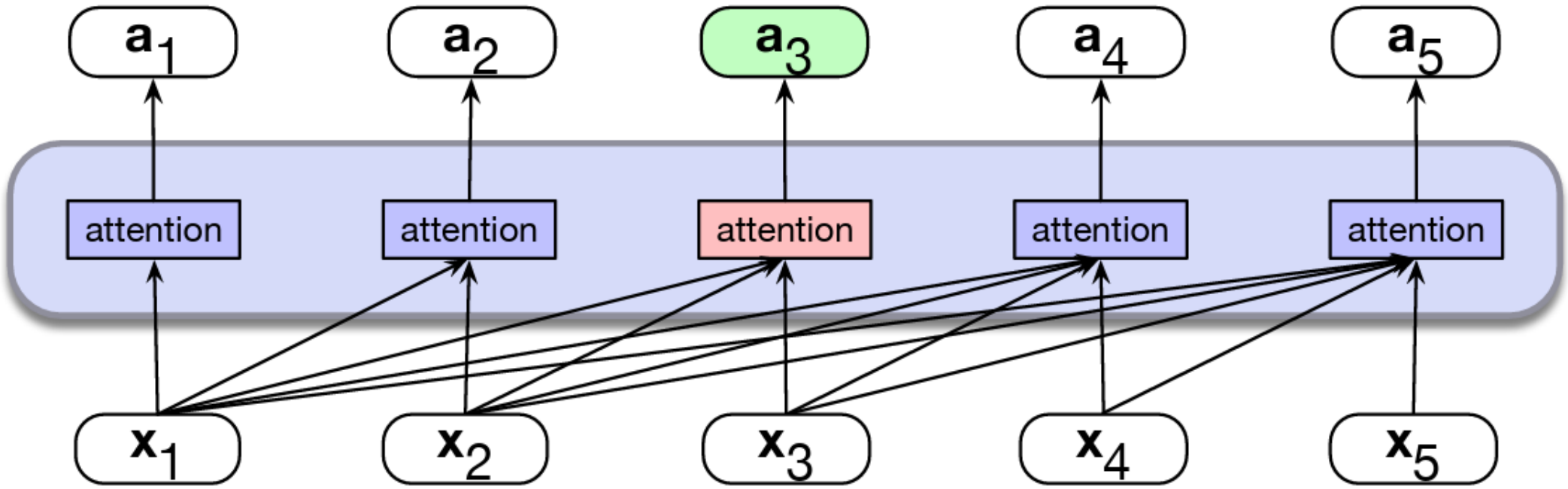
Encoder

MT and structure-
Based problems



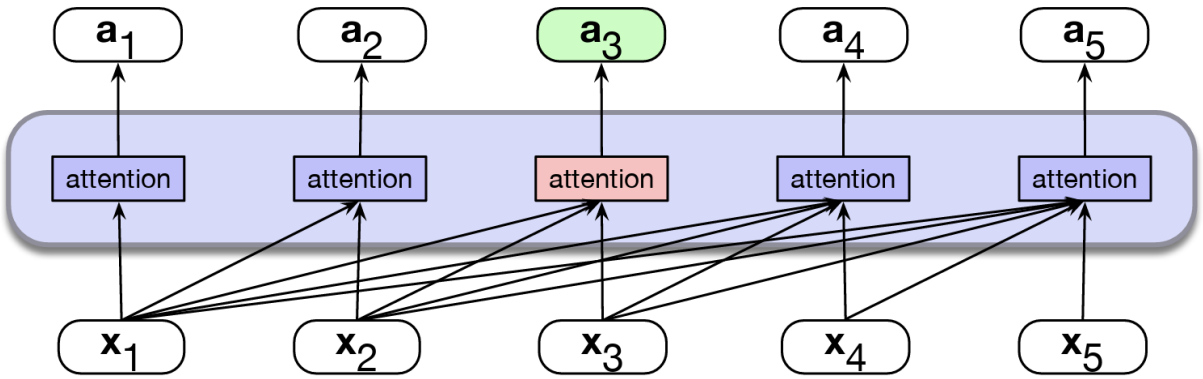
Encoder-Decoder

Causal Self-Attention Layer

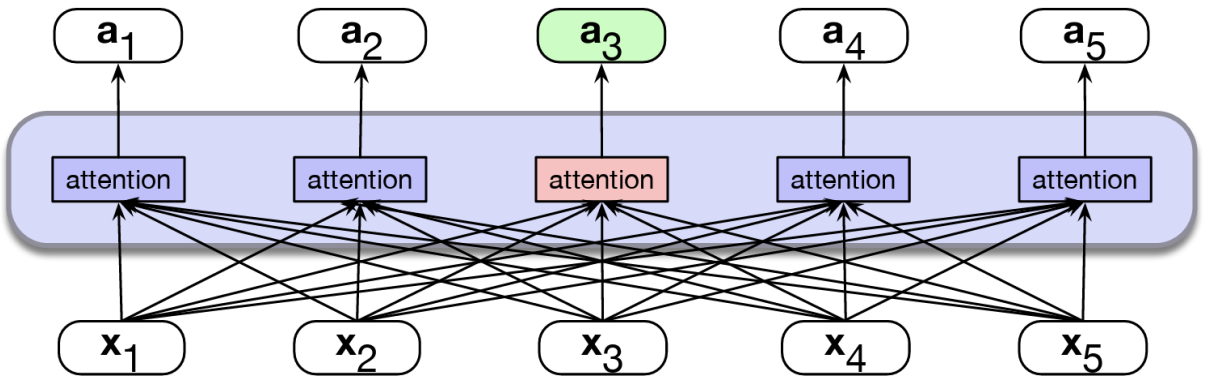


Why are we restricting ourselves to using the left

Bidirectional Transformer

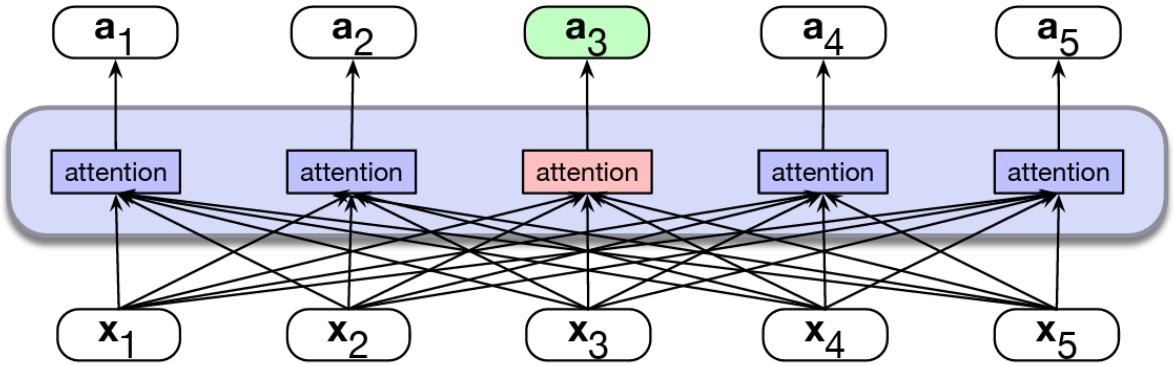


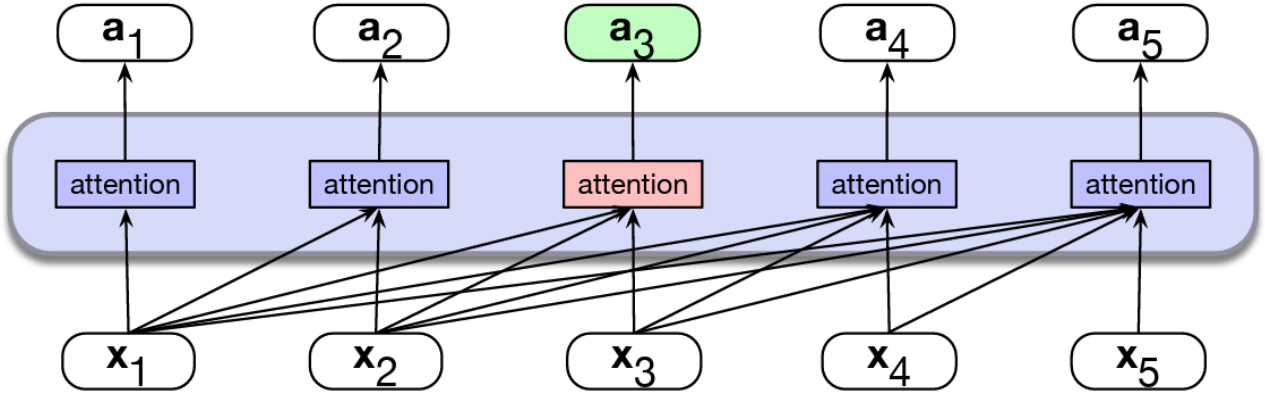
a) A causal self-attention layer



b) A bidirectional self-attention layer

Been There





N

q1•k1	q1•k2	q1•k3	q1•k4	q1•k5
q2•k1	q2•k2	q2•k3	q2•k4	q2•k5
q3•k1	q3•k2	q3•k3	q3•k4	q3•k5
q4•k1	q4•k2	q4•k3	q4•k4	q4•k5
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

N

N

q1•k1	−∞	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞	−∞
q4•k1	q4•k2	q4•k3	q4•k4	−∞
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

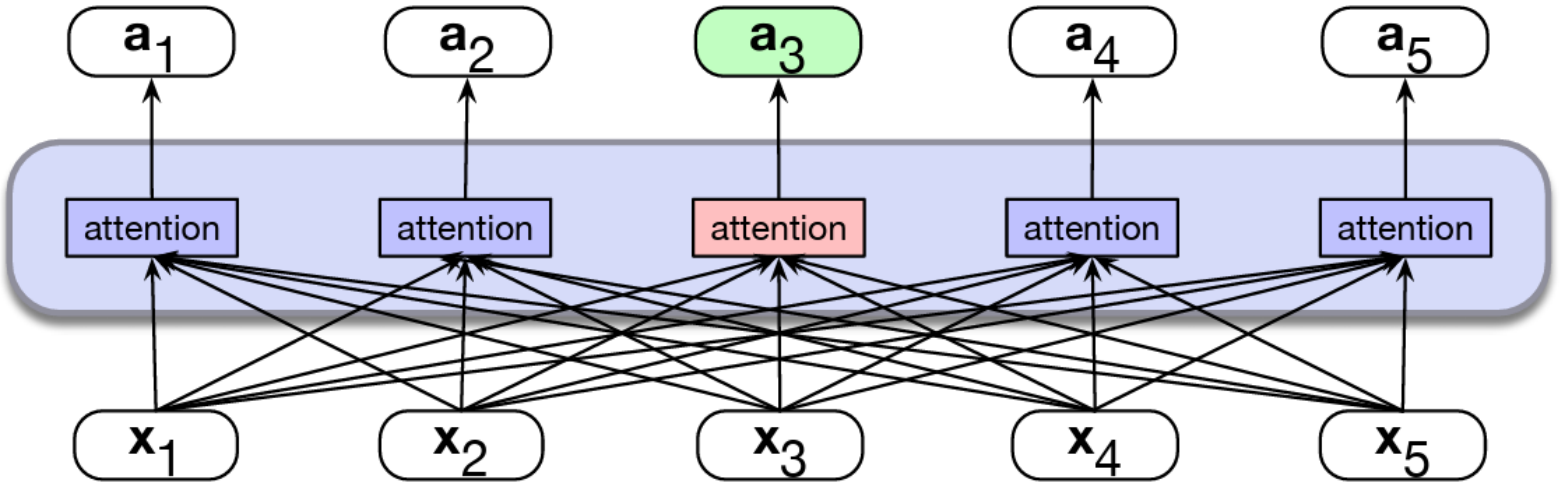
N

Why

- What does that get us?
  - Uses total context to capture an input. Not just the context to the left.
  - But we can't use it for autoregressive decoding
    - It's not actually a probabilistic language model
- Contextual Embeddings
  - Use these as a source of input for classifiers

Training

- But if we do this how do we train it to do language modeling? We can't use guess the next word as a training task since we already know it.



Alternative Training Tasks

- Still want the training to be self-supervised
  - Training objective should be based on the text itself
- Still want to use a prediction task
  - Either word prediction or some other prediction about the text itself

Cloze Task Inspiration

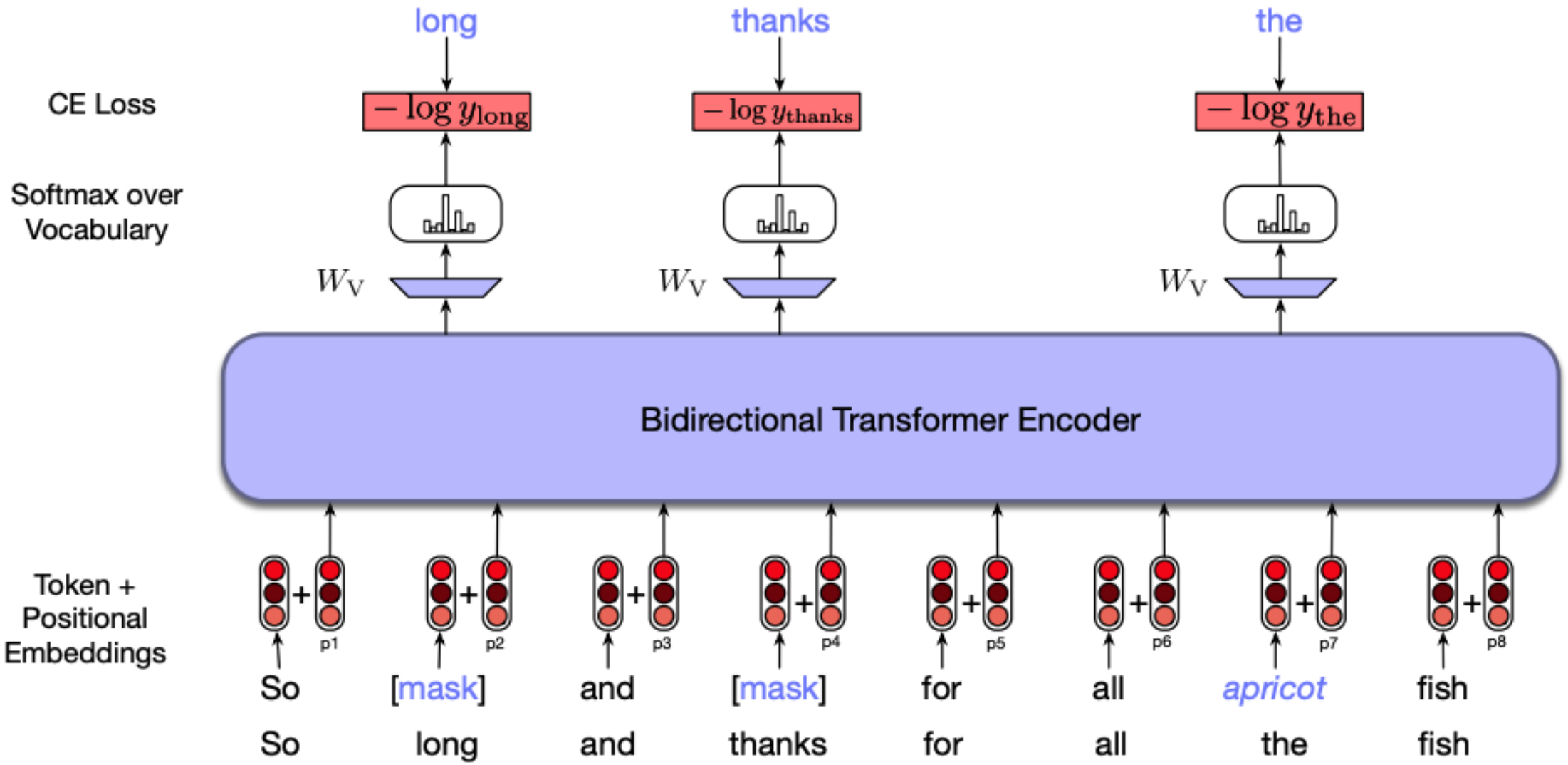
Traditional language modeling…

I notice three guys standing on the _____.

Cloze task (fill in the blank)

I notice three guys _____on the sidewalk.

Masked Language Model Training



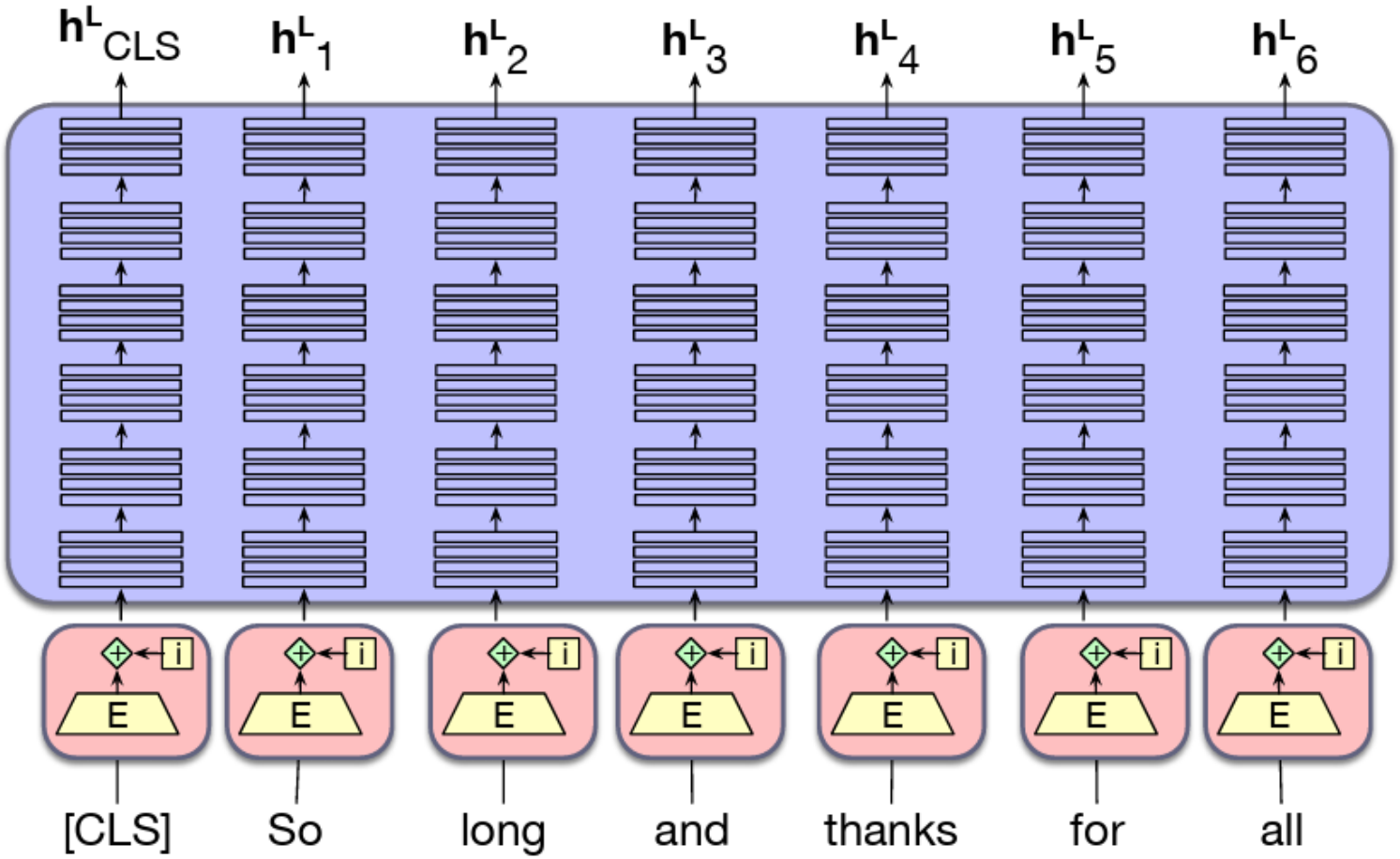
BERT MLM

- Given a training corpus, randomly select 15% of input tokens as training items
  - Replace input token with [MASK] (80%)
  - Replace input token with some other random word (10%)
  - Leave token as is.
- In all three cases, learning is based on CE loss from predicting the correct training item.

Contextual Embeddings

The embeddings from the last layer can be viewed as replacement for static embeddings (word2vec, Glove, LSA, etc).

Any application using those can use contextual embeddings as drop-in replacement



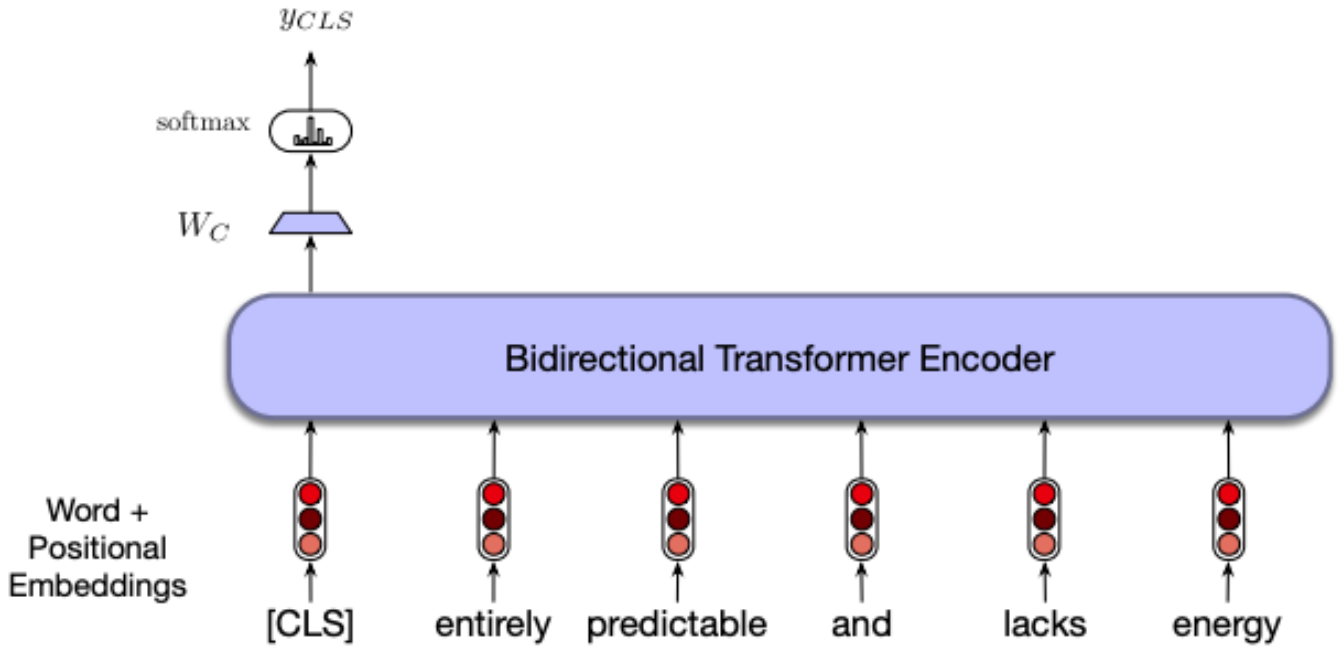
Finetuning Applications on BERT

Pretraining uses masking to train a language model from very very large collections of text.

- Finetuning consists of adding layers on the output for further application-specific processing
- Supervised training is used to learn the weights for the additional layers and (optionally) adjust the transformer weights to the application.

Sequence/Text Classification

1. Prepend a new token/embedding to the vocabulary.
   - [CLS] by convention.
   - Embedding is randomly initialized
2. During fine-tuning the contextualized [CLS] vector feeds a classifier.
   - Softmax classifier trained using CE loss
   - During fine-tuning the output of the other tokens is ignored.
3. Backprop the training loss into the BERT model and further into the word embeddings.



Sequence Classification Details

- Use the final hidden layer corresponding to the [CLS] token as our representation of the input sequence. Call that vector C. $C \in \mathbb{R}^H$
- Assume the classification task has K classes. Treat the problem as a multinomial logistic regression problem with weights over the elements of C.

$$W \in \mathbb{R}^{K \times H}$$

Sequence Classification Details

- To classify [CLS] into one of the K classes, compute the softmax over the possible classes

$$\text{softmax}(WC))$$

- W is K x H, C is H x 1. So WC is K x 1.

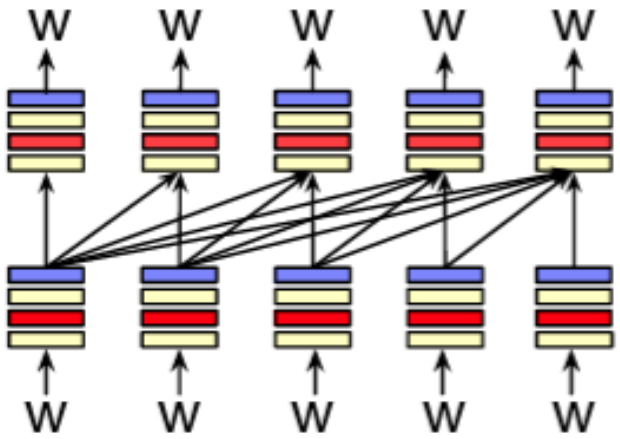
Sequence Classification Details

- Fine-tuning BERT for a classification task means
  1. learning W and
  2. adjusting the weights in the BERT model using a standard CE loss function
- Typically training proceeds over a few epochs. Enough to train W to adequate performance on the supervised task.
- The parameters in the BERT model are updated but not to a significant degree (or left frozen)

Architectures

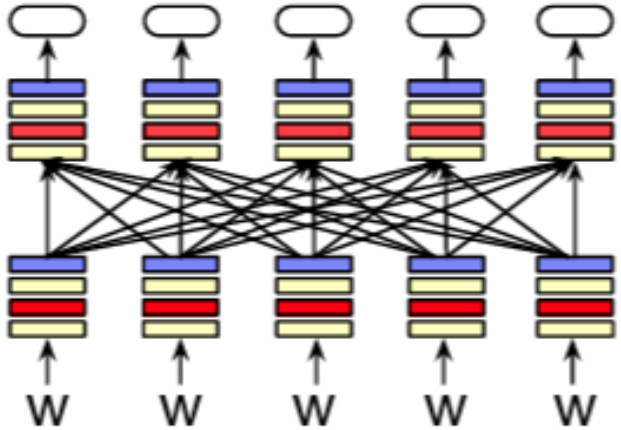
1. Language Models
2. Gradient-based learning
3. Embeddings

GPT family



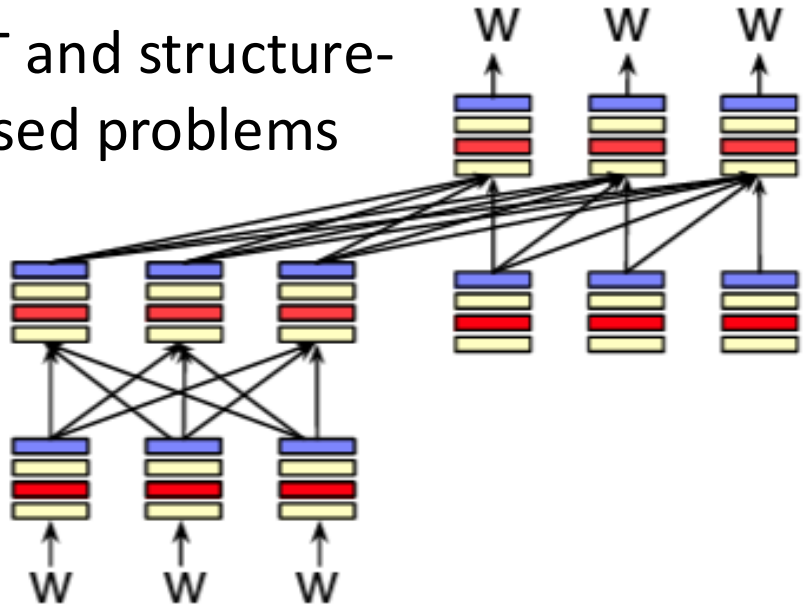
Decoder

BERT family



Encoder

MT and structure-
Based problems



Encoder-Decoder